Specification

Specification · Filter catalog: cheap AND in-stock · Build with Waqar

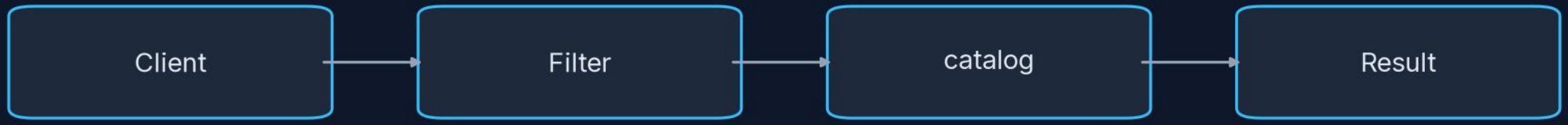
Today's Hook

- Problem: Filter catalog: cheap AND in-stock
- Pattern: Specification — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Filter catalog: cheap AND in-stock
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Specification: Filter catalog: cheap AND in-stock

Real-World Scenario

- Running example: Filter catalog: cheap AND in-stock
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.specification.problem;

import java.util.*;

public final class NaiveFilter {
  public static List<String> filter(List<int[]> items){ // [price, stock]
    List<String> out=new ArrayList<>();
    for (int i=0;i<items.size();i++) if(items.get(i)[0]<1000 && items.get(i)[1]>0) out.add("item"+i);
    return out;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.enterprise.specification.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record Product(String id, int priceCents, int stock){}
interface Spec { boolean isSatisfiedBy(Product p); default Spec and(Spec o){ return p -> this.isSatisfiedBy(p) && o.isSatisfiedBy(p); } }
final class CheapSpec implements Spec { private final int max; CheapSpec(int max){this.max=max;} public boolean isSatisfiedBy(Product p){ return p.priceCents()<max; } }
final class InStockSpec implements Spec { public boolean isSatisfiedBy(Product p){ return p.stock()>0; } }
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.specification.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Specification: cheap AND in-stock ===");
    System.out.println("--- naive ---"); System.out.println(NaiveFilter.filter(List.of(new int[]{500,1}, new int[]{2000,5}, new int[]{100,0})));
System.out.println("--- specification ---");
var catalog=List.of(new Product("a",500,1), new Product("b",2000,5), new Product("c",100,0));
Spec spec=new CheapSpec(1000).and(new InStockSpec());
System.out.println(catalog.stream().filter(spec::isSatisfiedBy).map(Product::id).toList());
  }
}
```

Walkthrough Output

```
=== Specification: cheap AND in-stock ===
--- naive ---
[item0]
--- specification ---
[a]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Specification.

Related Patterns

- Contrast Specification with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Specification → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.